

ПРОГРАМИРАНЕ (C++)

Лектор: доц. д-р Николай Касъклиев

Контакти kasakliev@uni-plovdiv.bg

УЧЕБНИ ДЕЙНОСТИ

- Лекции
- Упражнения
- Самостоятелна подготовка

Оценяване

- Изпит

ЛИТЕРАТУРА

- Крушков Х., Програмиране на C ++ Ч.1: Въведение в програмирането, ISBN 954561188X
- Наков и колектив, Основи на програмирането със C++, <https://cpp-book.softuni.bg/>
- Увод в програмирането със C++, <http://learn.dev.bg/%D1%83%D0%B2%D0%BE%D0%B4-%D0%B2-%D0%BF%D1%80%D0%BE%D0%B3%D1%80%D0%B0%D0%BC%D0%B8%D1%80%D0%B0%D0%BD%D0%B5%D1%82%D0%BE-%D1%81%D1%8A%D1%81-c/>
- <https://www.cplusplus.com/>
- Savitch, Walter J., Absolute C++ -- 5th ed., ISBN 978-0-13-283071-3
- Stanley B. Lippman, Josée Lajoie, Barbara E. Moo, C++ Primer Fifth Edition, ISBN 0-321-71411-3

ТЕМА 1. КОМПЮТЪРНИ СИСТЕМИ, ЕЗИЦИ ЗА ПРОГРАМИРАНЕ И КОМПЮТЪРНИ ПРОГРАМИ

УВОД

- Компютрите са навсякъде около нас
- Използваме ги за всякакви цели
- Улесняват почти всички от дейностите ни, вкл, обучение, бизнес, свободно време
- Относително достъпни са, но много от приложенията се заплащат
- Непрекъснато се усъвършенстват
- За да ги използваме за специфични наши нужди може да се наложи да създаваме приложения за тази цел
- Съществуват и множество рискове свързани с използването им

КОМПЮТЪРНА СИСТЕМА

Компютърна система = хардуер + софтуер

- **Компютърът е електронна машина (хардуер)**
 - Изграден е от множество компоненти (електронни и механични) всеки с конкретно предназначение
 - За да извърши дадена задача той трябва да бъде управляван
- **Софтуер – „мозъкът“ на машината**
 - Софтуерът служи да накара компютъра да извърши конкретна задача (например да изчисли среден успех, да намери максимална стойност, да сортира списък с имена)
 - Създава се от програмистите по специални правила
 - Има точно определена функционалност (възможности)

ДЕФИНИЦИИ

- **Език (естествен)**

Система от писмени символи или звуци, използвани за комуникация

- **Език за програмиране (ЕП) (*programming language*)**

(1) Система от знаци, използвана от човек за комуникация, която позволява детайлното записване на алгоритми по недвусмислен начин, така че те да могат да бъдат изпълнявани от компютър.

(2) Език за програмиране е система от символи, думи и словосъчетания за описание на изчисленията във форма, която може да се прочете и от компютър и от човек.

- **Изчисление (*computation*)**

Най-общо - всяко действие, извършвано от компютър (математически изчисления, обработка на изображения, текстообработка, ...).

- **Компютърна програма (*computer program*)**

Описание на изчисленията

КОМПЮТЪРНИ ПРОГРАМИ

Програмата специфицира обработката на данните

- За да може компютърът да изпълни определена обработка е необходимо да получи поредица от *разбираеми* команди
- За решаване на различните задачи се подават *различни поредици* от команди
- Поредицата от команди (наричат се още инструкции) за решаване на определена задача от компютъра се нарича *програма*
- Процесът на създаване на програми се нарича *програмиране*
- Множеството от команди (инструкции), с помощта на които се записват програмите формира т.нар. *език за програмиране*
- Всеки компютър има свой собствен език нарича се *машинен език*, дължи се на това, че всеки процесор(семејство процесори) имат различни т.нар. *instruction set*

КОМПЮТЪРНИ ПРОГРАМИ

Програмата представлява:

ПРОГРАМА=ДАННИ+АЛГОРИТЪМ

- Данни - от изключителна важност е характерът/същността на данните
- Алгоритъм – последователност от операции върху данните решаващи поставената задача

ДАНИИ

Дефиниции

Единица информация, факт, информация съхранявана в електронна форма, числа събирани с цел изследване или вземане на решение, информация с която работят компютрите

Тип/същност

Числови (парични суми, брой....)

- Символни (имена на стоки, кодове на стоки ...)
- Логически (да/не, пр. Пол)

Операции – в зависимост от типа (аритметични за числовите, сравнение на низове за символните...)

Преобразуване на типове – много често прилагана операция от програмиста

Дискусия с примери ...

АЛГОРИТЪМ

Дефиниции

- Рецепта
- Множество от стъпки за извършване на някаква задача
- Поредица от елементарни действия описващи изпълнението на дадена задача (изчисление)

Видове основни(базови) алгоритми

- *Линейни*
- *Разклонени*
- *Циклични*

Дискусия с примери ...

ЕЛЕМЕНТИ НА ЕП – ПРИМЕРИ

Всеки език, включително българския и машинния език има два аспекта – **синтаксис** и **семантика**, които го определят

- “**пвдсрг**” – не може да бъде дума от българския език, защото нарушава неговите правила за формиране на думи (не е синтактично правилна)
- “**бонижен**” – синтактично правилна, но семантично празна, защото не означава нищо
- “**Прозрачни автомобили спят зимен сън**” – изречение, което е синтактически правилно, но на практика е безсмислено

ХАРАКТЕРИЗИРАЩИ ЕЛЕМЕНТИ НА ЕП

- Както всеки писмен език, така и езикът за програмиране се основава на азбука:

Азбуката е винаги крайно множество от символи

- **Правилни изречения** на даден език представляват последователности от символите на азбуката, конструирани по т.нар. **синтактични правила** на езика (синтаксис, *syntax*)
- Правилните изречения написани на някой от езиците за програмиране се наричат **програми**
- Разбирането, което се влага в така конструираните правилни изречения, т.е. програмите и техните съставни части, се определя от т.нар. **семантични правила** на езика (семантика, *semantics*)
- Взети заедно, азбуката, синтаксисът и семантиката формират езика за програмиране (ЕП)

СИНТАКСИС И СЕМАНТИКА НА ЕП

- Строго определен набор от правила на ЕП
- Необходимост:
 - за да бъде програмистът сигурен в правилността и ефективността на програмата, която пише
 - програмата да бъде "разбрана" от компютър и от всеки, който я чете
- *Синтактичните* правила определят как:
 - програмите се изграждат като последователност от символи
 - думите (речникът) на езика могат да се обединяват, за да се получат правилни изречения (да се покаже взаимовръзката им/структурата на изречението)
- *Семантичните* правила:
 - приписват смисъл и значение на комбинациите от думи
 - **т.е. семантиката** е смисълът на градивните елементи на езика и връзката с означените обекти

Синтаксисът определя формата на програмите, а семантиката - техния смисъл

СИНТАКСИС И СЕМАНТИКА НА ЕП – ПРИМЕРИ

- *Синтаксис:*

- изречение, подлог, сказуемо, допълнение, предложен израз, съществително, глагол, прилагателно, и т.н. са синтактични концепции в българския език
- програма, начало, декларация, тяло, команда, израз, идентификатор и т.н. са синтактични концепции в програмните езици
- `int a = 2` е синтактично правилно твърдение на C++ (то спазва правилата за образуване на твърдения в C++)

- *Семантика:*

- `int a = 2;` за програмиста на C++ то означава да се намери ново място в паметта и да се запази в него числото две

ВИДОВЕ ЕЗИЦИ ЗА ПРОГРАМИРАНЕ

ЕП най-често се класифицират според:

- областта на приложение
- нивото на абстракция
- програмната парадигма заложена в основата на реализацията им

ВИДОВЕ ЕП СПОРЕД ОБЛАСТТА НА ПРИЛОЖЕНИЕ

- *Универсални ЕП* – пригодни за описание на произволни алгоритми ако всяка задача, която може да се реши с компютър, има някакво решение и което може да се изрази с този език
- *Специализирани ЕП* – създадени за специфичен вид обработка

Примери:

- XSLT (eXtensible Stylesheet Language Transformations) е език за трансформиране на XML документи
- SQL за управление на база данни
- Csound за звукови компилатори/синтезатори

ВИДОВЕ ЕП СПОРЕД НИВОТО НА АБСТРАКЦИЯ – I И II ПОКОЛЕНИЕ

Разглеждат се четири поколения ЕП, всяко от които е стъпка по посока на писането на програми чрез термини, които представят човешката интерпретация на задачата и резултатите

- **Първо поколение** (50-те) – машинни езици

Азбуката, синтаксисът и семантиката на машинните езици се определят изцяло от вида и начина избрани за вътрешното представяне на алгоритмите в компютъра. Всяка команда съответства пряко на едно-единствено действие на машината

- **Второ поколение** (до 1954) – езици за асемблиране (комплектоване)

- позволяват задаване на символни и относителни адреси, както и на потребителски дефинирани команди (*макроси*) в примитивна форма (напр. Add A B вместо 101101010011101001011010) . Програма, наречена *асемблер* превежда всеки символ от програмата на езика за асемблиране и връща символ от машинен език. Като се съберат всички символи от машинния език, асемблерът извежда програма на машинен език

ВИДОВЕ ЕП СПОРЕД НИВОТО НА АБСТРАКЦИЯ – III ПОКОЛЕНИЕ

- *Третото поколение* (1954 -1970) – ЕП от високо ниво

Абстракцията в техните семантични правила е доста развита, а синтаксисът им е близък до този на естествените език (напр. позволява да пишем изрази от типа $a = 1 + 1$)

Примери: FORTRAN I, Algol 58, Flowmatic (те са базирани на математически изрази), FORTRAN II (въвежда подпрограмите, отделното компилиране), ALGOL 60 (въвежда блоковата структура), COBOL (въвежда структурирането на данните, а обработката на файлове е вградена в самия език), APL (за научни изчисления), LISP (функционално програмиране за изкуствен интелект), PL/I (на IBM), Pascal (наследник на ALGOL 60), SIMULA (класове, абстракции от данни, обектно-ориентирано програмиране)

ВИДОВЕ ЕП СПОРЕД НИВОТО НА АБСТРАКЦИЯ – IV ПОКОЛЕНИЕ

- *Четвъртото поколение* (1970-80) – ЕП, които се характеризират с промяна в семантиката

Позволяват на програмистите да изразяват идеите си за нещата по начин естествен за човека, чрез понятия, които не съответстват на никоя компютрна концепция

Примери: Smalltalk (ООП), С (системно програмиране), Prolog (изкуствен интелект), Modula-2 (модули: капсулиране, скриване на информация, абстрактни типове данни, базиран на *обекти*), Ada (пакети: капсулиране, скриване на информация, абстрактни типове данни, поддръжка на паралелизъм), C++, Object-Pascal, Modula-3 (добавят *обектно-ориентирани* възможности към съществуващите езици).

Най-мощните от съществуващите езици за програмиране са Visual BASIC (офис приложения), Borland Delphi, Visual C++/ C# (езици за лесна разработка на десктоп приложения), Perl и Awk и Sed (текст), Tk-Tcl (скриптов езици), Java (десктоп, мобилни и уеб-програмиране)

ВИДОВЕ ЕП СПОРЕД ПАРАДИГМАТА НА ПРОГРАМИРАНЕ – ПРОЦЕДУРНО (ИМПЕРАТИВНО) ПРОГРАМИРАНЕ

Това е стандартният (традиционен) програмен стил, в който програмите се раздробяват на отделни стъпки (части, наречени *процедури*), чрез които се описват изчисленията

- **Характеристики** на процедурните ЕП:
 - изпълнението на програмите е последователно команда след команда
 - променливите представят определени места в паметта
 - за промяна стойността на променливите се използва операцията присвояване
- **Примери:** Pascal, C, C++, Perl, Python, Go

ВИДОВЕ ЕП СПОРЕД ПАРАДИГМАТА НА ПРОГРАМИРАНЕ – *ФУНКЦИОНАЛНО ПРОГРАМИРАНЕ*

Функционалният стил на програмиране се основава на теорията на математическите функции. Функциите са основните градивни блокове на програмата. Изпълнението на програмата представлява изчисляване на стойности чрез *изрази* и *функции*. Те могат свободно да бъдат предавани като параметри, както и да бъдат конструирани и връщани като параметри - резултат на други функции

- **Характеристики** на функционалните ЕП:
 - функциите се прилагат върху стойности, както и върху други функции за създаването на по-сложни функции
 - предаване на параметри
 - липсва понятието променлива
 - няма цикли, но широко се използва рекурсията
- **Примери:** Lisp, Miranda, ML, Haskell

ПРИМЕРНА ПРОГРАМА НА MIRANDA

```
output = title ++ captions ++ concat (map line [1..20])
```

```
title = cjustify 60 "A TABLE OF POWERS" ++ "\n\n"
```

```
captions = format "N" ++ concat (map caption [2..5]) ++ "\n"
```

```
caption i = format ("N^" ++ shownum i)
```

```
format = rjustify 12
```

```
line n = concat [format (show(n^i)) | i<-[1..5]] ++ "\n"
```

ВИДОВЕ ЕП СПОРЕД ПАРАДИГМАТА НА ПРОГРАМИРАНЕ – ЛОГИЧЕСКО (ДЕКЛАРАТИВНО) ПРОГРАМИРАНЕ

Този стил изисква декларативно описание на задачата, вместо задаване на отделните стъпки на алгоритъма на задачата. Декларативните програми са по-близки до спецификацията, отколкото традиционното програмиране

- **Характеристики** на логическите ЕП:
 - те са базирани на формалната логика
 - няма цикли
 - програмите се състоят от аксиоми (факти), правила за правене на заключения и цел
 - програмата описва кое е вярно по отношение на очаквания резултат
 - изпълнението на програмата представлява правене на извод
- **Примери:** Prolog, OPSS и Clips (езици базирани на правила)

ПРИМЕРНА ПРОГРАМА НА PROLOG

```
get_answer( A ) :-
```

```
    ratom( X ), name( X, String ),  
    valid_resp( String, A ), !.
```

```
valid_resp( [H|T], A ) :- type_ans( H, A ).
```

```
type_ans( X, A ) :- ([X] = "y"; [X] = "Y"), A = yes.
```

```
type_ans( X, A ) :- ([X] = "n"; [X] = "N"), A = no.
```

```
type_ans( X, A ) :- ([X] = "w"; [X] = "W"), A = why.
```

```
?-print("\nYou can start animal by typing "help.<CR>"\n' ).
```

```
valid_resp( [], A ) :-
```

```
    print("\nPlease try to give me a yes or no answer.'),  
    get_answer( A ), !.
```

```
valid_resp( [H|T], A ) :- valid_resp( T, A ).
```

ВИДОВЕ ЕП СПОРЕД ПАРАДИГМАТА НА ПРОГРАМИРАНЕ – ОБЕКТНО-ОРИЕНТИРАНО ПРОГРАМИРАНЕ

При обектно-ориентирания стил на програмиране се набляга на дефинирането на класове от обекти. Нужните екземпляри (обекти) от съответен клас се създават по време на изпълнението на програмата

- **Характеристики на обектно-ориентирания ЕП:**
 - обектът е съвкупност от клетки в паметта и операции, които променят стойностите в тях
 - най-простият обект е променливата
 - изчислението е взаимодействие и комуникация между обектите
 - всеки обект се държи като компютър – със собствена памет и операции
 - класът е група от обекти с еднакви свойства
 - обектите се създават като екземпляри на класовете
- **Примери:** Smalltalk, Eiffel, Java, C++, C#

ЕП И ПАРАДИГМИ НА ПРОГРАМИРАНЕ

<i>Процедурни/ императивни</i>	<i>Декларативни</i>		<i>Обектно- ориентирани</i>
	<i>Функционални</i>	<i>Логически</i>	
Machine code Assembly PL/I Pascal Modula-2 BASIC C C++, Java, Go Rython	Haskell ML LISP Miranda Logo APL F# Elm	PROLOG и производни Fril ALF	Smalltalk Simula C++ Objective-C Oberon Modula-3 PHP Ruby Java C#



Ще бъдат разгледани характеристиките и основните елементи на **универсалните процедурни ЕП от високо ниво**

ДОПЪЛНИТЕЛНО СЪДЪРЖАНИЕ

- По желание студентите могат да се запознаят допълнително с методи за описание на синтаксис и семантика на езиците за програмиране.
- Това допълнително съдържание няма да се включва в изпита.

МЕТОДИ ЗА ОПИСАНИЕТО НА СЕМАНТИКАТА НА ЕП

Ще представим два начина на описание на семантиката на ЕП:

- учебници-справочници за езика (с помощта на някакъв човешки език)
- формално дефиниране:
 - аксиоматична семантика (определят смисъла на команда в програма, като описва ефекта върху състоянието на програмата. Твърденията са логически - предикати с променливи, където променливите определят състоянието на програмата.)
 - операционна семантика (формализира, като описва значението като поредица от стъпки - изчисления)
 - денотационна семантика (още се нарича математическа формализира значението на ЕП с математически обекти)

При изучаване на даден ЕП семантичните му правила често се описват неформално, с изречения на естествен език

МЕТОДИ ЗА ОПИСАНИЕТО НА СИНТАКСИСА НА ЕП

Синтактични правила на един ЕП са сравнително малко и точно определени, ето защо е удобно те да се описват чрез строг и *формален език*

Най-известните методи за специфициране на синтаксиса на един програмен език са:

- форми на Бекус-Наур
- синтактични диаграми

Двата метода

- в основата си са еднакви, различават се само по възприетите начините за означение при задаване на синтактичните правила

- използват понятията:

- **терминален символ** – дума от речника на езика
- **нетерминален символ** – по-сложна синтактична конструкция, съставена от терминални символи и други синтактични конструкции. Нетерминалните символи трябва да бъдат заместени от терминални (според правилата), за да се получи правилна конструкция на ЕП

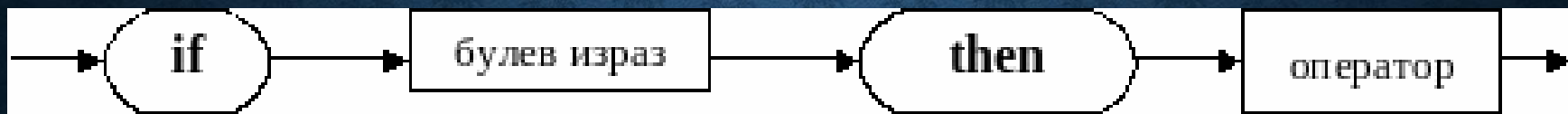
ОПИСАНИЕ НА СИНТАКСИСА НА ЕП ЧРЕЗ ФОРМИ НА БЕКУС-НАУР (БНФ)

Следният пример описва синтаксиса на реалните числа с цяла част, десетична точка и дробна част в езика Pascal. Пример.

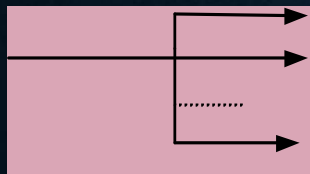
$\langle \text{реално-число} \rangle ::= \langle \text{последователност} \rangle . \langle \text{последователност} \rangle$
 $\langle \text{последователност} \rangle ::= \langle \text{цифра} \rangle | \langle \text{цифра} \rangle \langle \text{последователност} \rangle$
 $\langle \text{цифра} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$

- Създателите са Джон Бекус и Питър Наур. За първи път БНФ се използва при описанието на синтаксиса на Algol 60
- **Деривационният символ** от всяка продукция се записва $::=$
- **Нетерминални** символи – думи или фрази, записани в ъглови скоби ($\langle \dots \rangle$)
- **Терминални** символи се записват в кавички (“...”)
- Други **метасимволи** са:
 - $\{ \}$ – нула или повече повторения
 - $[]$ – част от синтаксиса, която не е задължителна
 - Символът $|$ е логическо “или”

ОПИСАНИЕ НА СИНТАКСИСА НА ЕП ЧРЕЗ СИНТАКТИЧНИ ДИАГРАМИ



- Графичен език за описанието на синтаксис на ЕП
- Над входната стрелка се записва името на синтактичната категория, която се определя с дадената синтактична диаграма
- **Нетерминални** символи – думи или фрази, записани в правоъг. блок () Например цифра или цяло_без_знак ...
- **Терминални** символи се записват в овали. Напр. if, then
-  – стрелката свързва отделните елементи на диаграмата и показва посоката на обхождане. Всяка диаграма има по една входна и изходна стрелка



– разклонение

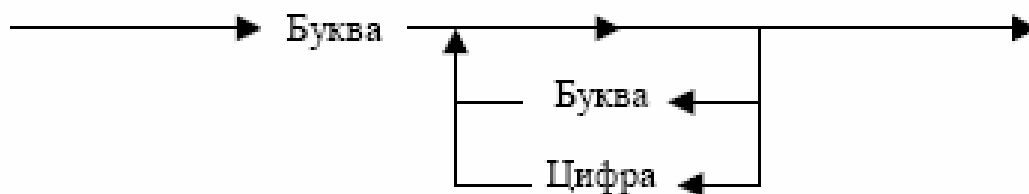


– цикъл

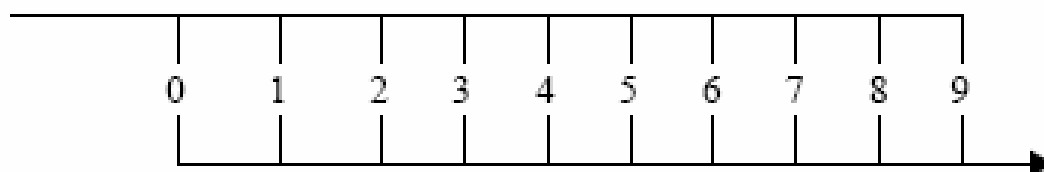
ОПИСАНИЕ НА СИНТАКСИСА НА ЕП ЧРЕЗ СИНТАКТИЧНИ ДИАГРАМИ

Синтаксис на идентификатор

Идентификатор::



Цифра::



Буква::

